Databases

TODAY'S AGENDA

Database Systems Background

Relational Model

Relational Algebra

Alternative Data Models

Q&A Session

DATABASE

Organized collection of inter-related data that models some aspect of the real-world.

Databases are the core component of most computer applications.

DATABASE EXAMPLE

Create a database that models a digital music store to keep track of artists and albums.

Information we need to keep track of in our store:
→ Information about Artists
→ The Albums those Artists released

FLAT FILE STRAWMAN

Store our database as comma-separated value (CSV) files that we manage ourselves in application code.

→ Use a separate file per entity.

→ The application must parse the files each time they want to read/update records.

Artist(name, year, country)

```
"Wu-Tang Clan",1992,"USA"
"Notorious BIG",1992,"USA"
"GZA",1990,"USA"
```

Album(name, artist, year)

```
"Enter the Wu-Tang","Wu-Tang Clan",1993
"St.Ides Mix Tape","Wu-Tang Clan",1994
"Liquid Swords","GZA",1990
```

FLAT FILE STRAWMAN

Example: Get the year that GZA went solo.

Artist(name, year, country)

```
"Wu-Tang Clan",1992,"USA"
"Notorious BIG",1992,"USA"
"GZA",1990,"USA"
```

```
for line in file.readlines():
  record = parse(line)
  if record[0] == "GZA":
    print(int(record[1]))
```

FLAT FILES: DATA INTEGRITY

How do we ensure that the artist is the same for each album entry?

What if somebody overwrites the album year with an invalid string?

What if there are multiple artists on an album?

What happens if we delete an artist that has albums?

FLAT FILES: IMPLEMENTATION

How do you find a particular record?

What if we now want to create a new application that uses the same database? What if that application is running on a different machine?

What if two threads try to write to the same file at the same time?

FLAT FILES: DURABILITY

What if the machine crashes while our program is updating a record?

What if we want to replicate the database on multiple machines for high availability?

DATABASE MANAGEMENT SYSTEM

A **database management system** (DBMS) is software that allows applications to store and analyze information in a database.

A general-purpose DBMS supports the definition, creation, querying, update, and administration of databases in accordance with some **data model**.

DATA MODELS

A **data model** is a collection of concepts for describing the data in a database.

A **schema** is a description of a particular collection of data, using a given data model.

→ This defines the structure of data for a data model.

→ Otherwise, you have random bits with no meaning.

DATA MODELS

Relational ← Most DBMSs

Key/Value

Graph

Document / JSON / XML / Object

Wide-Column / Column-family

Array (Vector, Matrix, Tensor)

Hierarchical

Network

Semantic

Entity-Relationship

DATA MODELS

Relational

Key/Value ← Simple Apps / Caching

Graph

Document / JSON / XML / Object

Wide-Column / Column-family

Array (Vector, Matrix, Tensor)

Hierarchical

Network

Semantic

Entity-Relationship

DATA MODELS

Relational

Key/Value

Graph

Document / JSON / XML / Object

Wide-Column / Column-family

← NoSQL

Array (Vector, Matrix, Tensor)

Hierarchical

Network

Semantic

Entity-Relationship

DATA MODELS

Relational

Key/Value

Graph

Document / JSON / XML / Object

Wide-Column / Column-family

Array (Vector, Matrix, Tensor) ← ML / Science

Hierarchical

Network

Semantic

Entity-Relationship

DATA MODELS

Relational
Key/Value
Graph
Document / JSON / XML / Object
Wide-Column / Column-family
Array (Vector, Matrix, Tensor)
Hierarchical
Network
Semantic
Entity-Relationship

← Obsolete / Legacy / Rare

RELATIONAL MODEL

The **relational model** defines a database abstraction based on relations to avoid maintenance overhead.

Key tenets:

→ Store database in simple data structures (relations).
→ Physical storage left up to the DBMS implementation.
→ Access data through high-level language, DBMS figures out best execution strategy.

RELATIONAL MODEL

Structure: The definition of the database's relations and their contents independent of their physical representation.

Integrity: Ensure the database's contents satisfy constraints.

Manipulation: Programming interface for accessing and modifying a database's contents.

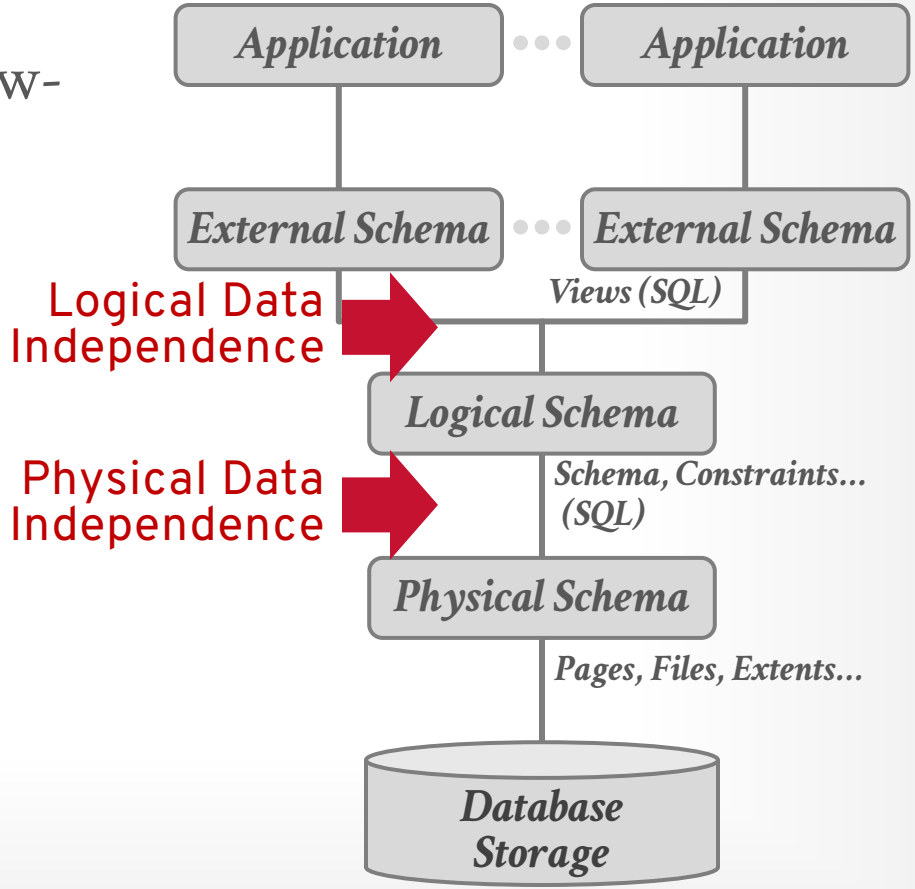
DATA INDEPENDENCE

Isolate the user/application from low-level data representation.

→ The user only worries about high-level application logic.

→ DBMS optimizes the layout according to operating environment, database contents, and workload.

→ DBMS can then re-optimize the database if/when these factors changes.



RELATIONAL MODEL

A **relation** is an unordered set that contain the relationship of attributes that represent entities.

A **tuple** is a set of attribute values (aka its **domain**) in the relation.

→ Values are (normally) atomic/scalar.

→ The special value **NULL** is a member of every domain (if allowed).

Artist(name, year, country)

name	year	country
Wu-Tang Clan	1992	USA
Notorious BIG	1992	USA
GZA	1990	USA

n-ary Relation

=

Table with *n* columns

RELATIONAL MODEL: PRIMARY KEYS

A relation's **primary key** uniquely identifies a single tuple.

Some DBMSs automatically create an internal primary key if a table does not define one.

DBMS can auto-generation unique primary keys via an identity column:

→ **IDENTITY** (SQL Standard)
→ **SEQUENCE** (PostgreSQL / Oracle)
→ **AUTO_INCREMENT** (MySQL)

Artist(id, name, year, country)

id	name	year	country
101	Wu-Tang Clan	1992	USA
102	Notorious BIG	1992	USA
103	GZA	1990	USA

RELATIONAL MODEL: FOREIGN KEYS

A foreign key specifies that an attribute from one relation maps to a tuple in another relation.

RELATIONAL MODEL: FOREIGN KEYS

Artist(id, name, year, country)

id	name	year	country
101	Wu-Tang Clan	1992	USA
102	Notorious BIG	1992	USA
103	GZA	1990	USA

ArtistAlbum(artist_id, album_id)

artist_id	album_id
101	11
101	22
103	22
102	22

Album(id, name, year)

id	name	year
11	Enter the Wu-Tang	1993
22	St.Ides Mix Tape	1994
33	Liquid Swords	1995

RELATIONAL MODEL: CONSTRAINTS

User-defined conditions that must hold for <u>any</u> instance of the database.

→ Can validate data within a single tuple or across entire relation(s).

→ DBMS prevents modifications that violate any constraint.

Unique key and referential (fkey) constraints are the most common.

SQL:92 supports global asserts but these are rarely used (too slow).

Artist(<u>id</u>, name, year, country)

id	name	year	country
101	Wu-Tang Clan	1992	USA
102	Notorious BIG	1992	USA
103	GZA	1990	USA

```
CREATE TABLE Artist (
 name VARCHAR NOT NULL,
 year INT,
 country CHAR(60),
 CHECK (year > 1900)
)
```

```
CREATE ASSERTION myAssert
 CHECK ( <SQL> );
```

DATA MANIPULATION LANGUAGES (DML)

The API that a DBMS exposes to applications to store and retrieve information from a database.

Procedural:

→ The query specifies the (high-level) strategy to find the desired result based on sets / bags.

← Relational Algebra

Non-Procedural (Declarative):

→ The query specifies only what data is wanted and not how to find it.

← Relational Calculus

RELATIONAL ALGEBRA

Fundamental operations to retrieve and manipulate tuples in a relation.

→ Based on set algebra (unordered lists with no duplicates).

Each operator takes one or more relations as its inputs and outputs a new relation.

→ We can "chain" operators together to create more complex operations.

σ	Select
π	Projection
∪	Union
∩	Intersection
–	Difference
×	Product
⋈	Join

RELATIONAL ALGEBRA: SELECT

Choose a subset of the tuples from a relation that satisfies a selection predicate.

→ Predicate acts as a filter to retain only tuples that fulfill its qualifying requirement.

→ Can combine multiple predicates using conjunctions / disjunctions.

Syntax: $\sigma_{predicate}(R)$

R(a_id,b_id)

a_id	b_id
a1	101
a2	102
a2	103
a3	104

$\sigma_{a_id='a2'}(R)$

a_id	b_id
a2	102
a2	103

$\sigma_{a_id='a2' \wedge b_id>102}(R)$

a_id	b_id
a2	103

```
SELECT * FROM R
 WHERE a_id='a2' AND b_id>102;
```

RELATIONAL ALGEBRA: PROJECTION

Generate a relation with tuples that contains only the specified attributes.

→ Rearrange attributes' ordering.
→ Remove unwanted attributes.
→ Manipulate values to create derived attributes.

Syntax: $\Pi_{A1,A2,\ldots,An}(R)$

R(a_id,b_id)

a_id	b_id
a1	101
a2	102
a2	103
a3	104

$\Pi_{b_id-100,a_id}(\sigma_{a_id='a2'}(R))$

b_id-100	a_id
2	a2
3	a2

```
SELECT b_id-100, a_id
  FROM R WHERE a_id = 'a2';
```

RELATIONAL ALGEBRA: UNION

Generate a relation that contains all tuples that appear in either only one or both input relations.

Syntax: (R ∪ S)

R(a_id,b_id)

a_id	b_id
a1	101
a2	102
a3	103

S(a_id,b_id)

a_id	b_id
a3	103
a4	104
a5	105

```
(SELECT * FROM R)
UNION
(SELECT * FROM S);
```

(R ∪ S)

a_id	b_id
a1	101
a2	102
a3	103
a4	104
a5	105

RELATIONAL ALGEBRA: INTERSECTION

Generate a relation that contains only the tuples that appear in both of the input relations.

Syntax: (R ∩ S)

R(a_id,b_id)

a_id	b_id
a1	101
a2	102
a3	103

S(a_id,b_id)

a_id	b_id
a3	103
a4	104
a5	105

(R ∩ S)

a_id	b_id
a3	103

```
(SELECT * FROM R)
INTERSECT
(SELECT * FROM S);
```

RELATIONAL ALGEBRA: DIFFERENCE

Generate a relation that contains only the tuples that appear in the first and not the second of the input relations.

Syntax: (R - S)

R(a_id,b_id)

a_id	b_id
a1	101
a2	102
a3	103

S(a_id,b_id)

a_id	b_id
a3	103
a4	104
a5	105

(R - S)

a_id	b_id
a1	101
a2	102

```
(SELECT * FROM R)
EXCEPT
(SELECT * FROM S);
```

RELATIONAL ALGEBRA: PRODUCT

Generate a relation that contains all possible combinations of tuples from the input relations.

Syntax: (R × S)

```
SELECT * FROM R CROSS JOIN S;
```

```
SELECT * FROM R, S;
```

R(a_id,b_id)

a_id	b_id
a1	101
a2	102
a3	103

S(a_id,b_id)

a_id	b_id
a3	103
a4	104
a5	105

(R × S)

R.a_id	R.b_id	S.a_id	S.b_id
a1	101	a3	103
a1	101	a4	104
a1	101	a5	105
a2	102	a3	103
a2	102	a4	104
a2	102	a5	105
a3	103	a3	103
a3	103	a4	104
a3	103	a5	105

RELATIONAL ALGEBRA: JOIN

Generate a relation that contains all tuples that are a combination of two tuples (one from each input relation) with a common value(s) for one or more attributes.

R(a_id,b_id)

a_id	b_id
a1	101
a2	102
a3	103

S(a_id,b_id,val)

a_id	b_id	val
a3	103	XXX
a4	104	YYY
a5	105	ZZZ

Syntax: (R ⋈ S)

R.a_id	R.b_id	~~S.a_id~~	~~S.b_id~~	S.val
a3	103	~~a3~~	~~103~~	XXX

(R ⋈ S)

a_id	b_id	val
a3	103	XXX

RELATIONAL ALGEBRA: JOIN

Generate a relation that contains all tuples that are a combination of two tuples (one from each input relation) with a common value(s) for one or more attributes.

Syntax: (R ⋈ S)

R(a_id,b_id)

a_id	b_id
a1	101
a2	102
a3	103

S(a_id,b_id,val)

a_id	b_id	val
a3	103	XXX
a4	104	YYY
a5	105	ZZZ

(R ⋈ S)

a_id	b_id	val
a3	103	XXX

```
SELECT * FROM R NATURAL JOIN S;
```

```
SELECT * FROM R JOIN S USING (a_id, b_id);
```

```
SELECT * FROM R JOIN S
    ON R.a_id = S.a_id AND R.b_id = S.b_id;
```

RELATIONAL ALGEBRA: EXTRA OPERATORS

Rename (ρ)

Assignment (R←S)

Duplicate Elimination (δ)

Aggregation (γ)

Sorting (τ)

Division (R÷S)

OBSERVATION

Relational algebra defines an ordering of the high-level steps of how to compute a query.

→ Example: $\sigma_{b_id=102}(R \bowtie S)$ vs. $(R \bowtie (\sigma_{b_id=102}(S))$

A better approach is to state the high-level answer that you want the DBMS to compute.

→ Example: Retrieve the joined tuples from **R** and **S** where **b_id** equals 102.

RELATIONAL MODEL: QUERIES

The relational model is independent of any query language implementation.

SQL is the *de facto* standard (many dialects).

```
for line in file.readlines():
  record = parse(line)
  if record[0] == "GZA":
    print(int(record[1]))
```

```
SELECT year FROM artists
 WHERE name = 'GZA';
```

DATA MODELS

Relational ← This Course

Key/Value

Graph

Document / JSON / XML / Object

Wide-Column / Column-family

Array (Vector, Matrix, Tensor)

Hierarchical

Network

Semantic

Entity-Relationship

DATA MODELS

Relational

Key/Value

Graph

Document / JSON / XML / Object ← Leading Alternative

Wide-Column / Column-family

Array (Vector, Matrix, Tensor) ← New Hotness

Hierarchical

Network

Semantic

Entity-Relationship

DOCUMENT DATA MODEL

A collection of record documents containing a hierarchy of named field/value pairs.

→ A field's value can be either a scalar type, an array of values, or another document.

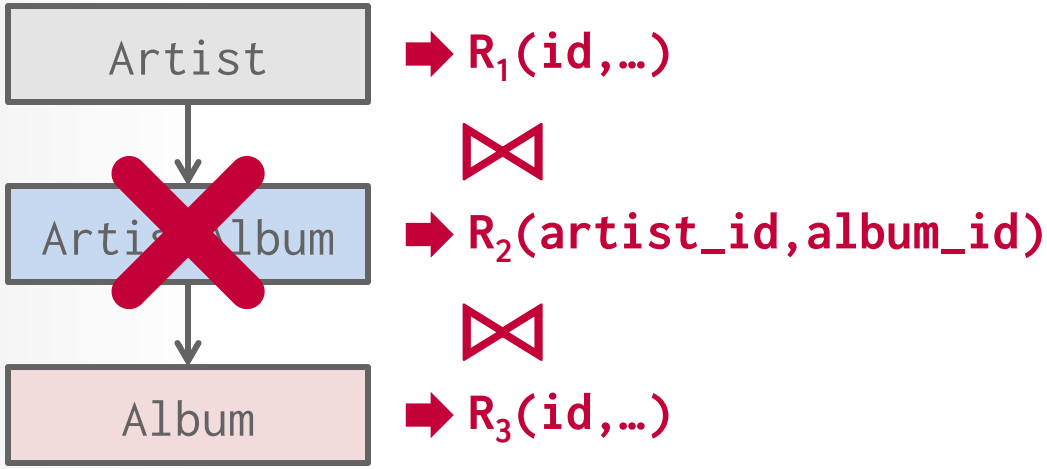
→ Modern implementations use JSON. Older systems use XML or custom object representations.

Avoid "relational-object impedance mismatch" by tightly coupling objects and database.

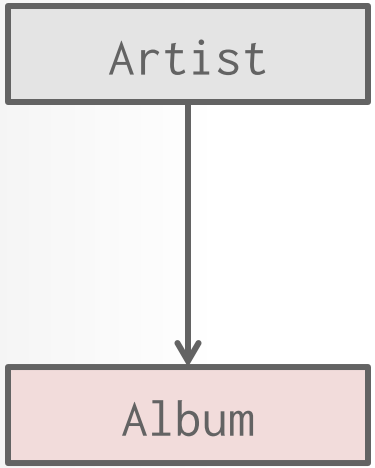
MongoDB. RAVENDB amazon DynamoDB Firebase

CouchDB relax Couchbase MarkLogic® fauna AEROSPIKE

DOCUMENT DATA MODEL



DOCUMENT DATA MODEL



Application Code

```
class Artist {
   int id;
   String name;
   int year;
   Album albums[];
}
class Album {
  int id;
  String name;
  int year;
}
```

```
{
 "name": "GZA",
 "year": 1990,
 "albums": [
   {
    "name": "Liquid Swords",
    "year": 1995
   },
   {
    "name": "Beneath the Surface",
    "year": 1999
   }
 ]
}
```

VECTOR DATA MODEL

One-dimensional arrays used for nearest-neighbor search (exact or approximate).

→ Used for semantic search on embeddings generated by ML-trained transformer models (think ChatGPT).

→ Native integration with modern ML tools and APIs (e.g., LangChain, OpenAI).

At their core, these systems use specialized indexes to perform NN searches quickly.

Pinecone Weaviate milvus qdrant

marqo CloseVector VectorDB turbopuffer <(°O°)>

VECTOR DATA MODEL

Album(id, name, year)

id	name	year
11	Enter the Wu-Tang	1993
22	St.Ides Mix Tape	1994
33	Liquid Swords	1995

Query

Find albums similar to "Liquid Swords"

OpenAI

Hugging Face

Transformer

[0.02, 0.10, 0.24, ...]

Embeddings

Id1 → [0.32, 0.78, 0.30, ...]

Id2 → [0.99, 0.19, 0.81, ...]

Id3 → [0.01, 0.18, 0.85, ...]

⋮

Vector Index

HNSW, IVFFlat

Meta Faiss, Spotify Annoy

Ranked List of Ids

CONCLUSION

Databases are ubiquitous.

Relational algebra defines the primitives for processing queries on a relational database.

We will see relational algebra again when we talk about query optimization + execution.